Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

Aim: To understand the process of convolution over the image and apply over the classification problem

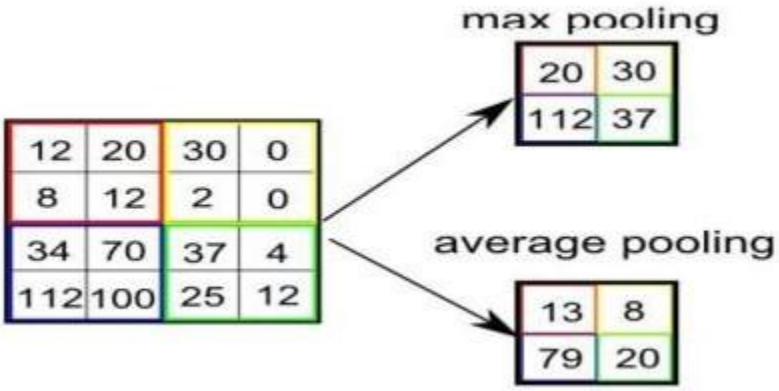
IDE: Google Colab

Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

What is Pooling Layer?

Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.



What is Convolutional Layer?

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

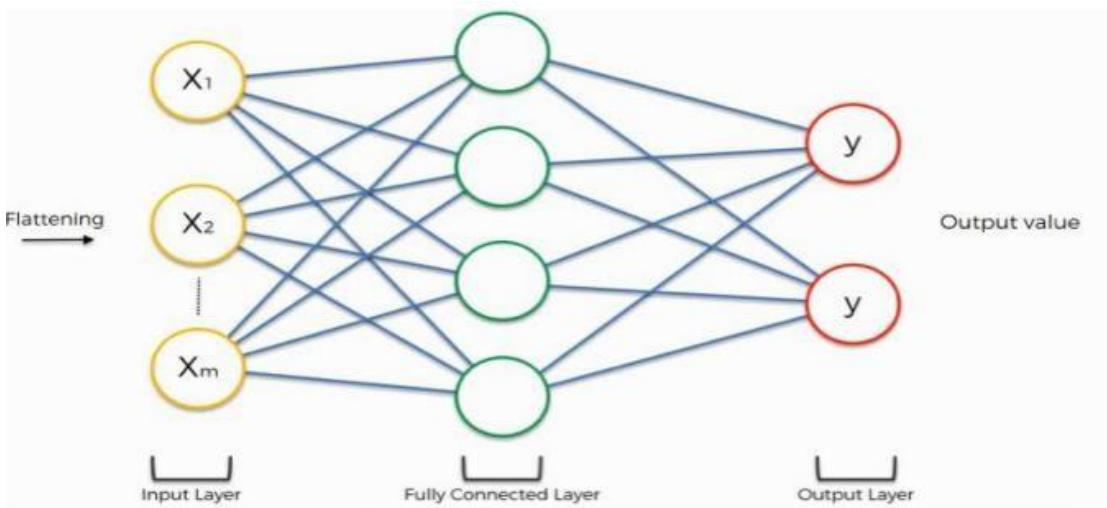
<u>What is Fully Connected Layer?</u>

A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

Input Layer: It takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

Fully Connected Layer: It takes the inputs from the feature analysis and applies weights to predict the correct label.

Output Layer: It gives the final probabilities for each label.



ReLU Layer: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data
7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

Program (Code):

```
# 1. Load the basic libraries and packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import keras
from keras.datasets import fashion_mnist
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

# 2. Load the dataset
# Fashion MNIST dataset (images + labels)
(train_X,train_Y),(test_X,test_Y)=fashion_mnist.load_data()
```

```
# 3. Analyse the dataset
print(train_X.shape,train_Y.shape)
print(test_X.shape, test_Y.shape)
print("Unique classes:", np.unique(train_Y))
```

```
# 4. Normalize the data
# Convert pixel values from (0-255) to (0-1)
train_X = train_X.astype('float32') / 255
test_X = test_X.astype('float32') / 255
```

```
# 5. Pre-process the data
# Reshape data for CNN input (28x28x1)
train_X = train_X.reshape(-1,28,28,1)
test_X = test_X.reshape(-1,28,28,1)
```

```
# Convert labels into one-hot encoding
train_Y = to_categorical(train_Y)
test_Y = to_categorical(test_Y)
```

```
# Split into training and validation sets
train_X,valid_X,train_label,valid_label = train_test_split(
    train_X,train_Y,test_size=0.2,random_state=13
)
```

```
# 6. Visualize the Data
# Show one sample image
plt.imshow(train_X[0].reshape(28,28))
plt.title("Sample Image")
plt.show()
```

```
# 7. Write the CNN model function
model = tf.keras.Sequential()
```

```
# Convolution Layer 1
model.add(tf.keras.layers.Conv2D(32,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Convolution Layer 2
model.add(tf.keras.layers.Conv2D(64,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Convolution Layer 3
model.add(tf.keras.layers.Conv2D(128,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Fully connected layers
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.Dense(10,activation='softmax'))
```

```
# 8. Write the Cost Function
# Using categorical crossentropy loss function
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

```
# 9. Gradient Descent Optimization Algorithm
# Adam optimizer is used (advanced form of gradient descent)
```

```
# 10. Apply training to minimize loss
history = model.fit(train_X,train_label,
                    batch_size=64,
                    epochs=20,
                    validation_data=(valid_X,valid_label))
```

```
# 11. Observe cost function vs iterations (learning curve)
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(len(loss))
```

```
plt.plot(epochs, loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Cost Function vs Iterations")
plt.legend()
plt.show()
```

Results:

To be attached with

a. Training dataset

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

b. Model summary

```
(60000, 28, 28) (60000,)
```

```
(10000, 28, 28) (10000,)
```

```
[0 1 2 3 4 5 6 7 8 9]
10
```

```
(60000, 28, 28, 1) (60000,)
(10000, 28, 28, 1) (10000,)
```

c. Training and validation accuracy w.r.t epochs before regularization

```
testing_evaluation=model.evaluate(test_X,test_Y_one_hot)

313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - accuracy: 0.9190 - loss: 0.4888
```

d. Training and validation loss w.r.t epochs before regularization

```
[0.4649391174316406, 0.9186000227928162]
```

e. Training and validation accuracy w.r.t epochs after regularization

```
{'accuracy': [0.8299583196640015,
 0.895270824432373,
 0.9107083082199097,
 0.9233124852180481,
 0.9338958263397217,
 0.9395208358764648,
 0.9474375247955322,
 0.9545000195503235,
 0.9603750109672546,
 0.9676250219345093,
 0.9725416898727417,
 0.9761458039283752,
 0.9793333411216736,
 0.9816041588783264,
 0.9852708578109741,
 0.9854791760444641,
 0.9859583377838135,
 0.9865208268165588,
 0.9897500276565552,
 0.9901041388511658],
```

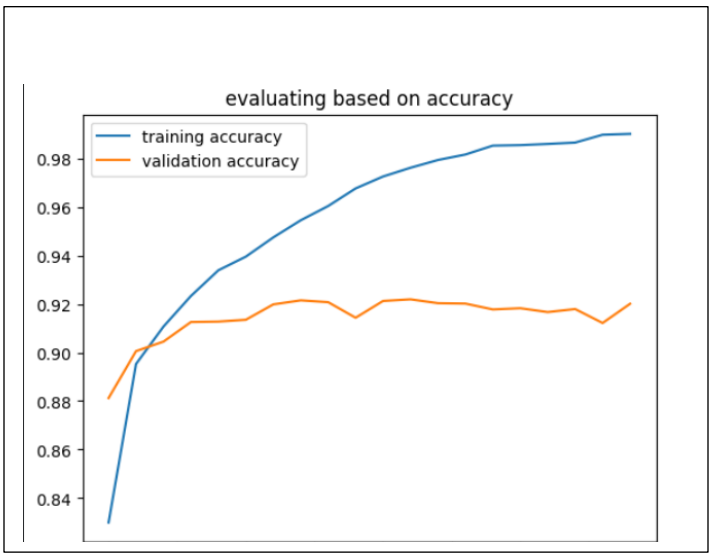
```
'val_accuracy': [0.8812500238418579,
 0.9005833268165588,
 0.9045000076293945,
 0.9125833511352539,
 0.9127500057220459,
 0.9135000109672546,
 0.9198333621025085,
 0.921500027179718,
 0.920750021934509 3,
 0.9143333435058594,
 0.9212499856948853,
 0.921916663646698,
 0.9203333258628845,
 0.9201666712760925,
 0.9177500009536743,
 0.9182500243186951,
 0.9166666865348816,
 0.9179166555404663,
 0.9121666550636292,
 0.9200833439826965],
```

```
'loss': [0.4644168019294739,
 0.2880558371543884,
 0.24071426689624786,
 0.2095048576593399,
 0.18173784017562866,
 0.16245578229427338,
 0.14074471592903137,
 0.12101194262504578,
 0.10491971671581268,
 0.08772961795330048,
 0.07507806271314621,
 0.06349695473909378,
 0.055666692554950714,
 0.04887472465634346,
 0.04083728417754173,
 0.03952484577894211,
 0.03677994757890701,
 0.03608689457178116,
 0.027847640216350555,
 0.027848264202475548],
```

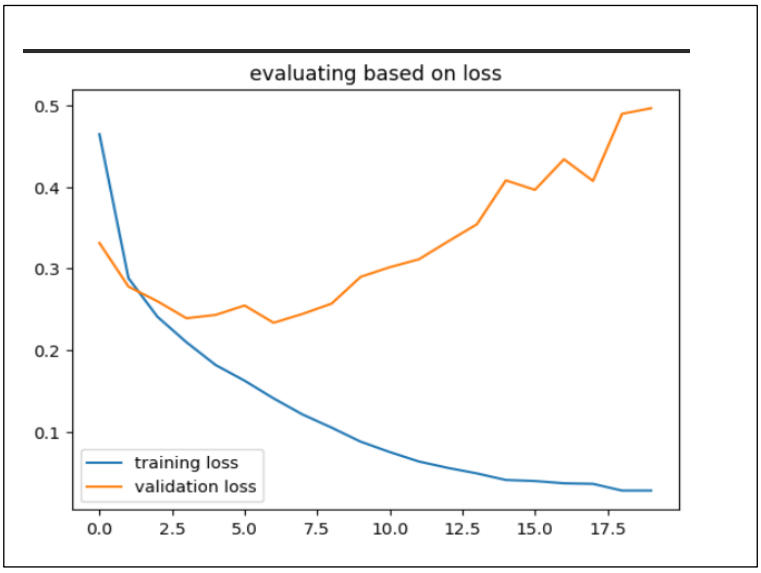
```
'val_loss': [0.331199049949646,
 0.2777387797832489,
 0.25971725583076477,
 0.23893453180789948,
 0.2430487424135208,
 0.2545752227306366,
 0.23344311118125916,
 0.24431003630161285,
 0.2569504678249359,
 0.2897614538669586,
 0.30149340629577637,
 0.31112468242645264,
 0.33286920189857483,
 0.35406702756881714,
 0.4078773558139801,
 0.3962751030921936,
 0.43371260166168213,
 0.40715593099594116,
 0.48940134048461914,
 0.49626782536506653]}
```

f. Training and validation loss w.r.t epochs after regularization

g. Original v/s predicted labels for correct predicted observations



h. Original v/s predicted labels for incorrect predicted observations



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

Pre Lab Exercise:

a. What is Convolution process? Explain giving example.

Convolution is an operation in which a small filter (kernel) moves across an image to extract meaningful features such as edges, textures, or patterns. For instance, in an image of a shirt, convolution can highlight edges, shapes, and textures by applying filters to the pixel values.

b. What are different layers in CNN model?

A CNN model is composed of multiple layers including Convolutional layers for feature extraction, Activation layers (such as ReLU) to introduce non-linearity, Pooling layers for reducing dimensions, Fully Connected layers for classification, and an Output layer for final predictions.

c. What is the requirement of the pooling layer in the CNN model?

Pooling reduces the size of feature maps, which helps in lowering computation and preventing overfitting. It also keeps important features while removing unnecessary details, making the model more efficient.

d. What is the requirement of the use of ReLU activation function after convolution step?

ReLU adds non-linearity to the model, enabling it to learn complex patterns. Without it, the network would behave like a linear model and would not effectively capture intricate features in images.

Observation and Result Analysis:

a. Nature of the dataset

The dataset used is Fashion MNIST, consisting of grayscale images of size 28×28 pixels that represent various clothing items. It includes 10 distinct classes and is well-organized for image classification tasks. The data is preprocessed through normalization and reshaping to suit CNN input requirements.

b. Training Process without regularization

Without regularization, the CNN model learns rapidly, and training accuracy increases consistently. However, validation accuracy does not improve at the same pace, indicating overfitting where the model starts memorizing the training data.

c. After regularization in the training Process

With the inclusion of dropout layers, the model generalizes better. Although training accuracy may slightly decrease, validation accuracy improves, reflecting better performance on unseen data and reduced overfitting.

d. Observation over the Learning Curves

The learning curves show that training loss decreases continuously, while validation loss stabilizes after regularization. The gap between training and validation curves reduces, indicating improved model generalization and stability.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92301733057**

Post Lab Exercise:

a. Why CNN is preferred over ANN for images

CNN is preferred for image-related tasks because it can automatically learn spatial features such as edges and patterns through convolution operations. It also uses fewer parameters compared to ANN, making it more efficient for processing image data.

b. Can CNN be applied over Text data? If yes, then how. If no, then why?

Yes, CNN can be applied to text data. Words are first converted into numerical representations (embeddings), and convolution operations are then used to capture patterns such as phrases or significant word combinations.

c. What is the role of dropout layer?

Dropout helps prevent overfitting by randomly deactivating a subset of neurons during training. This encourages the network to learn more generalized and robust features rather than relying on specific neurons.

d. What will happen if maxpooling is replaced with minpooling?

Replacing max pooling with min pooling would cause the model to focus on the smallest values instead of the most prominent features. This can lead to loss of critical information and negatively impact the model's performance.